

# Object-Oriented Programming and Data-Oriented Design: A Performance Comparison

Video Demo: <https://youtu.be/b8FbDDmuNIA>

Samuel Christian Brumskill  
Newcastle University  
Newcastle Upon Tyne / UK  
[S.Brumskill2@newcastle.ac.uk](mailto:S.Brumskill2@newcastle.ac.uk)

## **Abstract**

This dissertation investigates the performance of Object-Oriented Programming (OOP) and Data-Oriented Design in the context of game engine performance and game engine system performance. The comparison involves comparing data layouts of Object-Oriented Programming, Array of Structures (AoS), and Structure of Arrays (SoA). An existing Object-Oriented game engine in C++ has been converted to AoS and SoA, with the main differences in how data is accessed, laid out in memory, and owned. The algorithms are equivalent across the implementations without retaining the same physical design as each other.

The three implementations were benchmarked using the same entity properties, scene layout, and object counts. The workload ranges from 101 objects to 40001 objects on two different systems. The two systems contain a Ryzen 9 7900X and an Intel Core i7-13650HX. Performance was evaluated using frame time, frame rate, physics timing, and rendering timing.

The results show a small difference in performance when comparing AoS and SoA, with SoA consistently outperforming AoS across the different workloads and showing better physics system performance. At larger workloads, SoA has a bigger difference in frame time when compared to lighter workloads. OOP showed much bigger frame time and physics time when compared to AoS and SoA; this became more apparent as the workloads became bigger. OOP also couldn't complete workloads past 2501 objects, meaning that results could not be recorded for larger workloads. In an additional experiment, unused data was added to the AoS game object and the SoA game object; SoA reduced the average frame time by approximately 53.5% because the unused arrays were not accessed, whereas the additional data increased the stride of the AoS records.

The results suggest that data layout and access have a substantial impact on performance in the context of game engines, with AoS and SoA consistently showing much lower frame times and physics times compared to OOP, which eventually could not complete larger workloads. This influence is particularly apparent at larger workloads. However, each workload was recorded once, meaning

that results are descriptive and do not establish statistical repeatability.

## **1. Introduction**

### **1.1 Background**

Modern computer hardware has evolved substantially over the past two decades. Graphics processors have developed highly parallel architectures, while CPU performance has increasingly depended on multiple cores, deeper cache hierarchies, SIMD instruction sets and complex memory systems rather than clock-frequency growth alone [6, 8, 9]. Consequently, effective use of the memory hierarchy and good data locality have become important influences on software performance [8–10]. Modern game engines place substantial demands on the CPU because systems including physics, artificial intelligence, animation, scripting, scene management and rendering preparation must be processed every frame [4]. As game worlds increase in scale and complexity, efficiently processing large collections of data becomes increasingly important. Although algorithmic complexity remains a major consideration, physical data layout and memory-access patterns can also affect performance because cache-line utilisation, prefetching and memory latency influence the cost of repeated operations [10, 16, 21].

Object-Oriented Programming (OOP) is widely used in game development and commercial game engines because it supports abstraction, modularity, extensibility and code reuse [1–3, 4, 5]. Object-oriented software is commonly organised as collections of interacting objects that combine state with associated behaviour through mechanisms such as encapsulation, inheritance and polymorphism [1–3]. However, OOP does not prescribe a particular physical data layout. Related objects and components may be allocated separately and connected through pointers, which can introduce indirection and irregular memory-access patterns [5, 10, 11, 16]. When large collections of objects are processed repeatedly, these access patterns may reduce spatial locality and increase the cost of retrieving data through the cache hierarchy [10, 16]. Therefore, the maintainability benefits of

object-oriented abstractions may coexist with memory layouts that are less suitable for some data-intensive workloads.

Data-Oriented Design (DOD) is a software-design approach that begins with the data processed by a program, including its quantity, lifetime, structure and access frequency [7, 23, 24]. Rather than modelling the implementation primarily around real-world entities, DOD considers how data is organised and transformed by each system [7]. It commonly favours representations that reflect actual access patterns, including contiguous arrays for operations that repeatedly process similar values [5, 16, 17]. Such layouts may reduce pointer traversal and increase the proportion of useful data transferred in each cache line. **However**, their effectiveness remains dependent on the workload and the fields being accessed [11, 13, 17]. These principles are particularly relevant to game-engine systems that process large collections of transforms, physics properties and rendering data. Unity's Data-Oriented Technology Stack provides one commercial example of applying data-oriented and entity-component-system principles to data-intensive workloads [15]. As processor performance increasingly depends on parallelism, vectorisation and effective use of the memory hierarchy, understanding how data layout affects game-engine workloads has become an important performance consideration [8, 21, 22].

## 1.2 Problem Statement

Although DOD has gained attention in performance-aware programming, OOP remains widely used in commercial game engines [4, 5]. Existing research demonstrates that data-oriented techniques can improve performance in suitable workloads. Still, the size of the improvement depends on the algorithm, the fields accessed, the selected data layout and the hardware platform [11, 15, 17, 20]. Many existing studies either isolate individual processing kernels or evaluate broader ECS and DOTS architectures that also introduce changes such as archetype storage and multithreading [13–15, 17, 18]. Consequently, relatively few empirical studies directly compare OOP, AoS and SoA implementations of equivalent rendering and physics workloads within the same C++ game engine. This makes it difficult to separate the effects of physical data layout from wider architectural changes. This dissertation addresses that gap by implementing and benchmarking three functionally equivalent engine architectures under consistent workloads.

## 1.3 Aim

The main aim of this dissertation is to investigate the performance characteristics of Object-Oriented Programming, Array of Structures (AOS), and Structure of Arrays (SOA) by benchmarking these different approaches under identical workloads within common game engine systems. This dissertation seeks to determine how different approaches to data organisation influence CPU performance and to evaluate

the advantages and disadvantages of each paradigm within the context of modern game engine development.

## 1.4 Objectives

The Objectives of this dissertation are:

- To review the literature relating to Object-Oriented Programming, Data-Oriented Design, cache locality, cache hierarchy, and game engine architecture.
- To design and implement functionally equivalent game engines using OOP, AOS, and SOA using previous coursework (CSC8503) as the foundation for an AOS and SOA conversion.
- To benchmark each of these approaches using identical workloads and test conditions.
- To evaluate performance of the different approaches using frame rate and frame time, and subsystem execution times.
- To analyse the relationship between memory layout, cache locality, and game engine architecture.
- To identify the advantages and disadvantages of the different approaches.

## 1.5 Research Questions

The research questions this dissertation addresses are:

- How does Data-Oriented Design differ from Object-Oriented Programming in terms of the performance of a game engine?
- How does the different data layout of Array of Structures and structure of arrays affect the performance of a game engine?
- Which game engine systems benefit from adopting Data-Oriented Design?

## 2. Background and Related Studies

### 2.1 Object-Oriented Programming

Object-Oriented Programming (OOP) is widely used in game programming and commercial game engine development [4, 5]. Its adoption is commonly associated with its emphasis on abstraction, modularity and code reuse. C++ is a multi-paradigm language but directly supports object-oriented programming through classes, inheritance, virtual functions and abstract base classes [2]. These mechanisms enable encapsulation, abstraction, inheritance and run-time polymorphism. Stroustrup explains that “the fundamental idea in defining a new type is to separate the incidental details of the implementation (e.g., the layout of data used to store an object of the type) from the properties essential to the correct

## Object-Oriented Programming and Data-Oriented Design: A Performance Comparison

use of it (e.g., the complete list of functions that can access the data)” [2, p. 450]. This separation allows implementation details to be hidden behind a defined interface. Consequently, an object-oriented program may be organised as a collection of interacting objects that combine state with the operations associated with that state.

These principles help explain why OOP remains widely used in games programming. Game engines are composed of numerous systems that work together to produce the final application. These systems include rendering, physics, artificial intelligence, networking, animation and audio [6]. Martin argues that software should remain easy to change as it evolves [3, pp. 14–15]. More specifically, he presents polymorphism as a powerful architectural mechanism because it enables control of source-code dependencies through abstract interfaces [3, pp. 43–47]. A high-level component can therefore call lower-level functionality without depending directly on its concrete implementation. Instead, the implementation component depends upon an abstraction defined by the higher-level policy. This relationship allows the source-code dependency to point in the opposite direction to the run-time flow of control.

This argument is further supported by Gamma et al., who recommend programming to interfaces rather than implementations and favouring object composition over class inheritance [1, pp. 18–20]. Programming through interfaces reduces a component’s dependence upon concrete classes, while composition allows behaviour to be combined without tightly coupling subclasses to the implementation details of their parent classes. These practices can support reusable and extensible software designs. C++ provides mechanisms such as inheritance, abstract base classes and virtual functions for implementing class hierarchies and interchangeable interfaces [2].

Although OOP can support reusable and maintainable software, particularly as systems evolve, the paradigm does not prescribe a particular physical data layout. Object-oriented implementations can store data contiguously, but they may also represent relationships through pointers and allocate objects separately in memory. When large collections of objects are processed repeatedly, pointer indirection and scattered memory allocation can reduce spatial locality and increase cache misses [10, 11, 16]. These memory-access patterns may reduce CPU performance in data-intensive systems such as rendering and physics simulation. The relationship between physical data layout, memory-access behaviour and processor performance therefore motivates the consideration of Data-Oriented Design.

### **2.2 Data-Oriented Design**

Data-Oriented Design is a software design approach that focuses on the data processed by the program and examines the shape and transformations it undergoes within the program. This contrasts with Object-Oriented implementations: instead of modelling a problem around the world through a collection of interacting objects, Data-Oriented Design considers the quantity, structure, lifetime, and access frequency of the data before selecting an appropriate representation. Fabian argues that this means the problem domain isn’t mapped to the data [7, pp. 5-9], allowing programmers to focus on how the data is accessed, transformed, and laid out. This means that the data is treated as an important design decision rather than an implementation detail.

Fabian also argues that data should be considered in terms of its type, frequency, quantity, shape, and probability; this is because these characteristics influence the most appropriate representation and algorithm [7, pp. 10-11]. Fabian also argues that Data-Oriented Design is better suited to change because data can change. This is because in Object-Oriented Programming, data is tied to the object, so it can only change from the object’s perspective rather than from the source of the data. This makes it difficult to see how data can change. In Data-Oriented Design, it’s much easier to couple and decouple data and systems, meaning that data can be viewed from multiple viewpoints and changed based on its source [7, pp. 12-15].

Performance-aware programmers also support this perspective on Data-Oriented Design. Mike Acton illustrates the three lies of Object-Oriented programming: software is a platform, code is designed around a model of the world, and code is more important than data [23]. Hardware is the platform, not software; software executes on physical hardware, and the processor, memory hierarchy and data-access pattern therefore influence the cost of a transformation. [23].

Modelling the problem around the world is a mistake because it leads to monolithic, unrelated data structures and transforms since hiding data is implicit in world modelling [23]; this ties back into what Fabian said about leaving the problem domain in the design document and not mapping it to the data [7].

The only purpose of any code is to transform data; the programmer’s job is to be responsible for the data, not the code [23]. This means that to understand and solve problems, programmers must understand the data. This is also related to Fabian’s argument that software is all about data and data transformation, with these transformations performed around the hardware rather than on a world model [7, pp. 2-4].

Acton argues that these three lies lead to poor performance, poor concurrency, poor scalability of optimisations, poor stability, and poor testability. Acton argues that these are

problems that are inherent to a fully Object-Oriented approach to programming [23]. Many of these problems stem from the belief that software can be future-proof; this isn't possible because hardware is constantly changing, and therefore the problem is constantly evolving [23].

Andrew Kelley applies these principles to a practical example. Kelley illustrates the importance of memory layout and memory alignment; he argues that data doesn't need to be tied to an object and stored together physically [24]. Data can instead be grouped according to how frequently it is accessed and which operations consume it [24]. This means that frequently accessed data can be grouped contiguously, and infrequently accessed data can be stored separately. This type of organisation can reduce the amount of irrelevant data brought into the cache during processing.

This approach is also relevant to games, as game engines and games, by extension, perform similar operations on large collections of data. Nystrom argues that data locality is important for game performance; effective use of data locality depends on the physical layout of data in memory [6]. Conventional Object-Oriented implementations allocate data separately, connecting them through pointers, which leaves the memory allocator to control the physical layout of data in memory. Nystrom argues that components belonging to the same system can be placed contiguously in arrays and processed sequentially. This reduces the amount of pointer traversal and increases the likelihood that the data needed by consecutive iterations is located close together in memory [6].

The performance motivation for Data-Oriented Design is also supported by research into cache-conscious data structures. Chilimbi, Hill, and Larus identified that poor cache locality is a major cause of memory-access cost in pointer-manipulating programs [16]. The paper demonstrates that carefully placing data elements that are accessed together can improve spatial and temporal locality. The paper describes clustering related elements into the same cache blocks, reducing cache misses [16]. This paper shows that physical data layout in memory is just as important as the algorithms used within the program.

Although Data-Oriented Design is positioned as an alternative to Object-Oriented Programming, it doesn't mean that all aspects of Object-Oriented Programming should be discarded, and it can coexist with Data-Oriented Design. Fabian acknowledges that Object-Oriented Programming can also exist with Data-Oriented Design [7]. Nystrom also acknowledges that the focus on data locality and contiguous memory layouts can come at the cost of abstraction due to the lack of polymorphism.

## 2.3 Computer Architecture

Computer Architecture outlines how the CPU interacts with and processes data for use in software. Modern computers

organise storage into a hierarchy called the cache hierarchy. The hierarchy describes the relationships among the different forms of memory and the advantages and disadvantages of each. The higher the memory on the hierarchy, the faster it is, but it's also smaller and more expensive. The lower the memory on the hierarchy, the slower it is, but the larger and cheaper it is to produce.

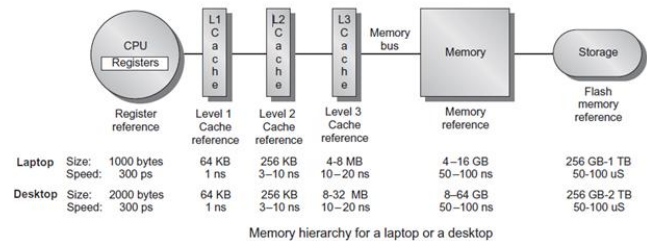


Figure 2.1: Cache Hierarchy for laptops or desktops [8, p.79]

L1 Cache is the fastest but smallest, with main memory being the slowest but largest [8, Ch 2; 9, Ch 6]. Cache memory essentially acts as an intermediate store that retains recently accessed or frequently used data close to the processor. This means that a program's performance depends not only on the number of instructions it executes but also on where its data is located within the memory hierarchy.

Two main factors can influence how effectively this hierarchy is used. Temporal locality refers to the repeated use of the same data within a short period. Spatial Locality refers to the tendency to access data stored at nearby memory addresses [9, Section 6.2]. When a load instruction requests data that is not present in the L1 cache, the processor attempts to obtain the corresponding cache line from a lower cache level or, if necessary, main memory. This means that the data may need to be retrieved from main memory. This is a problem, as main memory is substantially slower than the L1, L2, or L3 caches [10, pp. 13-16, 57-58]. Drepper argues that memory-access optimisations should focus on improving spatial and temporal locality while also aligning data properly [10, p. 48].

Data is transferred through the cache hierarchy in fixed-size blocks called cache lines. By accessing a single value, neighbouring bytes are also transferred into the cache. The usefulness of this depends on how the data is laid out in memory. If data is scattered in memory, the fetched cache line may contain neighbouring bytes that are not required by the current operation. Conversely, when values required by consecutive iterations are stored contiguously, a larger proportion of each fetched cache line may contain data useful to the current operation. This can improve spatial locality compared with access patterns that follow unrelated pointers or repeatedly skip over unused fields [6]. Intel's optimisation guidance clarifies that spatial locality refers to adjacent memory locations, including values stored within the same cache line [21, Section 9.2]. AMD's Zen 5 documentation

provides an example showing that data cache lines are 64 bytes wide; however, this varies across different CPUs [22, Section 2.6].

Modern CPUs use prefetching to reduce the visible cost of memory access. A hardware prefetcher attempts to recognise consistent patterns in memory access; this allows data to be retrieved before the program explicitly requests it. Drepper explains that the prefetcher can recognise sequential and fixed-stride access patterns, whereas irregular and non-repeating access patterns are harder to recognise [10, pp. 59-60]. Intel similarly recommends arranging data access into regular patterns so that the hardware prefetcher can overlap memory transfers with useful computation [21, pp. 9-1-9-3]. For these reasons, contiguous data layouts are preferable, as they are more predictable than following unrelated pointers.

Modern processors include SIMD instruction-set extensions that can increase computational throughput by applying the same operation to multiple packed values. Single Instruction, Multiple Data (SIMD) instructions allow a single instruction to be executed on multiple data items simultaneously [8, Ch. 4]. Intel identifies loops operating on sequential arrays as suitable candidates for SIMD execution; this is ideal for workloads that include processes such as rendering [21, pp. 5-13-5-14]. However, Effective vectorisation depends upon factors including regular memory access, alignment, aliasing, control flow and independence between loop iterations. This means that a program could be highly parallelizable while failing to use SIMD effectively if the data must be gathered from scattered locations.

These architectural characteristics complement game engines and games in general because games are data-intensive software. Different systems in games such as rendering and physics perform similar operations on large collections of transforms, positions, velocities, and collision properties. Differences in approach among Object-Oriented Programming, Array of Structures, and Structure of Arrays may alter the amount of useful data in a single cache line, how easily future accesses can be prefetched, and whether loops can be vectorised. The physical data layout in memory may affect performance even when the high-level algorithms remain equivalent throughout the different implementations.

## 2.4 Memory Layout (AoS and SoA)

The physical layout of data in memory determines which values move up the cache hierarchy via cache lines. There are two common layouts for storing data in memory, Array of Structures (AoS) and Structure of Arrays (SoA).

An Array of Structures layout stores each element as a structure containing all the properties associated with that element; these complete structures are stored consecutively in

memory. For example, an array of physics bodies will store their associated properties, such as inverse mass, position, velocity, and orientation, together.



Figure 2.2: Data Layout for AoS [13]

A Structure of Arrays layout stores each property in its own contiguous array; this means that properties such as position will be stored in an array containing only positions. These two layouts can have equivalent logical information but have different access patterns [17, 21].

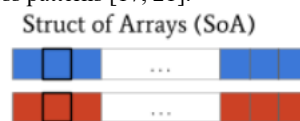


Figure 2.1: Data-Layout for SoA [13]

An array of Structures may use the cache inefficiently because data is transferred in cache lines, so when one member of an AoS element is accessed, the cache fetches the line containing that member. The same line may also contain other fields belonging to the structure or neighbouring elements. If the current loop does not use those fields, part of the transferred cache capacity may provide no immediate benefit. This means that values that may be unnecessary for computation could be sent through the cache hierarchy. As the structure becomes larger, fewer useful values can fit onto the cache line [8, Appendix B; 11, Secs. 6.2-6.4].

Intel explains that an AoS representation may require additional instructions and may underutilise SIMD lanes for operations that process the same field across multiple elements [21, Sec. 5.5.1]. Intel also distinguishes AoS as a horizontal computational model, in which different fields within a single structure are processed together [21, Sec. 7.5.1].

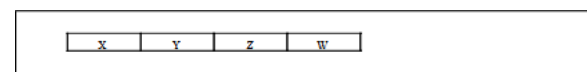


Figure 2.4: Horizontal Computation [21, Sec. 7.5.1]

SoA layouts instead place values representing the same property contiguously. This means that all position properties can pass through the cache hierarchy without sending other properties, such as orientation, inverse mass, or rendering data. This creates a unit-stride access pattern in which successive values are stored at consecutive or regularly spaced addresses.

Hennessy and Patterson describe unit-stride access patterns as the simplest memory access pattern for vector processing because cache blocks naturally contain several consecutive elements [8, Sec. 4.2]. In contrast, non-unit-stride and gather operations require values to be accessed from separate addresses and may use cache lines inefficiently [8, Sec. 4.2].

SoA layouts can be used to leverage SIMD instructions. SIMD is an instruction that performs a single operation on multiple data items; this requires the operands to be organised in a form suitable for loading into vector registers [8, Sec. 4.3]. Intel states that SoA is more suitable for vertical computation, where the same operation is applied to the same field across consecutive elements [21, Secs. 5.5.1 and 7.5.1].

|          |    |    |    |    |       |    |
|----------|----|----|----|----|-------|----|
| Vx array | X1 | X2 | X3 | X4 | ..... | Xn |
| Vy array | Y1 | Y2 | Y3 | Y4 | ..... | Yn |
| Vz array | Z1 | Z2 | Z3 | Y4 | ..... | Zn |
| Vw array | W1 | W2 | W3 | W4 | ..... | Wn |

**Figure 2.5: SoA Form of Representing Data [21, Sec. 7.5.1]**

A physics integration loop that applies the same calculations to multiple velocities or positions may benefit from vectorisation when those properties are stored contiguously. However, SoA doesn't guarantee that SIMD will be used; vectorisation also relies on alignment, aliasing, control flow, and whether loop iterations can be shown to be independent [8, Sec. 4.5].

Relationships between different data layouts and performance are also shown in studies, although no single layout is universally optimal. Homann and Laenen compare AoS and SoA using a particle-integration workload. Their results found that AoS performance deteriorates as additional unused properties are added to each particle. In their SoA implementation, unused properties are stored in separate arrays and don't affect performance because they aren't loaded by the integration loop [17, Sec. 2]. This is relevant to game physics simulations, since the integration operates on large collections of properties such as velocity and position; however, the workload of a game engine means that the numerical results won't translate to this study.

The significance of data layouts doesn't just pertain to comparisons between SoA and AoS. Hirzel evaluates ten different data layouts across 32 programs and three hardware configurations. The study found that cache and translation lookaside buffer miss rates commonly changed by approximately 10-20% between layouts. Execution time changed approximately 5-10%, with larger differences in some cases [11, pp. 265-266]. The results were also good and poor depending on the program. This indicates that there isn't a universally optimal layout and that it depends more on workloads; values should be arranged according to their relationships with the data accessed by the system [11, p. 275].

Data layouts must also be considered alongside the order in which the data is processed. Kandemir, Ramanujam and Choudhary show that cache locality can be improved through a combination of loop transformations and data-layout transformations [12]. Their work shows that changing the structure of the loop alone won't greatly improve performance. However, changing the data layout affects all operations that access that array. Although this study doesn't concern the development or performance of a game engine, the underlying principles remain relevant: effective locality depends on the relationship between physical data placement and traversal order, rather than on either factor in isolation.

## 2.5 Existing Performance Studies

Källman [13] reports a more recent experiment comparing Object-Oriented Design and Data-Oriented Design. The study evaluates control flow organisation and AoS versus SoA memory layouts using execution time and processor hardware counters. The memory layout experiments include scenarios that access every field, access a subset of fields, use padded fields and write to a single field. This is relevant because it recognises that the benefit of data layouts depends on the values the workload reads and writes, rather than treating all options as identical [13, Sections 4.3 and 5.3].

The relative difference between SoA and AoS remained within approximately  $\pm 16\%$  across the tested layout scenarios, and one test produced an unexpected result when only subfields were accessed. The results showed that SoA performance deteriorated as the number of elements increased [13, pp. 22-23]. There were more consistent results when the data exceeded the capacity of the last-level cache, where the benefits of SoA were shown. At this point, AoS showed higher cache misses, while cache-miss rate and execution time showed a positive correlation. The results are described as partially inconclusive: SoA showed clearer benefits when memory throughput became limited, but the results for data already residing within the cache were workload-dependent [13, pp. 22-26].

Research for Data-Oriented Design has also been performed in commercial game engines. Martin et al. [16] compared a Unity flight engine simulation using Unity's Data-Oriented Technology Stack and Unity's legacy Object-Oriented approach. The DOTs implementation uses Entity Component Systems and leverages multithreading to spawn and manipulate multiple aircraft. The study shows that CPU performance, frame rate, and memory usage were all better with the Data-Oriented implementation than with the Object-Oriented implementation. The study does show the advantages of Data-Oriented Design but also uses an Entity Component System and multi-threading; this makes it hard to isolate the extent of improvement attributable specifically to AoS or SoA.

## Object-Oriented Programming and Data-Oriented Design: A Performance Comparison

Research has also examined Entity Component Systems and their potential impact on performance. Tasnim and Zhao [14] provide a more direct comparison between Object-Oriented Programming and Data-Oriented Design by implementing a tower defence game. The study primarily focuses on Entity Component Systems, in which entities are identifiers, components contain data, and systems process entities by matching combinations of components. Structural changes, such as adding and removing components, cause entities to migrate between archetypes and are delayed until they can be processed safely [14, Sec 3-5].

The study compares the implementation of OOP, AoS-based ECS, a single-threaded SoA ECS, and a multithreaded SoA extension [14, Sections 6-7]. The study shows that the average FPS for OOP was 36.13; for AoS ECS, 61.51; for SoA ECS, 65.81; and for the parallel SoA, 109.54. This study is relevant because it shows single-threaded performance, thereby isolating the advantages of data layouts on performance while also demonstrating the benefits of multithreading. The slight improvement from AoS to SoA also shows that the conversion doesn't necessarily produce the same scale of performance increase from an Object-Oriented implementation.

Although this study is valuable, it has some flaws, including a lack of variation in testing. The study only compares workload at maximum capacity; it doesn't have different variations in enemy count, for example. Furthermore, the parallel SoA results are mainly attributable to multi-threading, not necessarily to the data layout itself.

There is also research investigating whether the exploration of data layouts can be automated. Chen, Varbanescu, and Naumann [18] demonstrate, using several benchmarks, that AoS and SoA choices can affect cache utilisation, automatic vectorisation, and false sharing. They propose using a C++ reflection system to preserve the original syntax while allowing changes to the memory layout [18, pp. 47-53]. This work is relevant as it identifies that many systems are deeply Object-Oriented and shows that converting these systems can be time-consuming and lead to differences in algorithm implementation. However, the benchmarks demonstrate that changing data layout can affect cache utilisation, automatic vectorisation and false sharing. However, the results are specific to the selected programs and should not be interpreted as evidence that one layout is universally superior. However, this paper is still a work in progress, which means it's more appropriate as evidence of an emerging research direction rather than as the main empirical support for a performance conclusion.

Mei and Tian [20] compare five different data layouts – SoA, AoS, aligned AoS, a combined SoA/AoS layout, and a hybrid layout – across three implementations of GPU-accelerated

inverse-distance-weighted interpolation. Their results don't show one universally optimal layout: AoS and aligned AoS performed better in the naïve and tiled implementations, whereas SoA performed best in the improved dynamic-parallelism implementation [20]. The combined layout showed no notable advantage when compared to the other, simpler layouts. This study shows that layout performance is influenced by the algorithm and execution strategy. However, this study mainly examines the effectiveness of these layouts on the GPU; it isn't representative of the performance of CPU-bound tasks such as physics and CPU-based rendering.

These studies show the advantages of Data-Oriented Design and how it can improve performance; however, they also highlight the difficulty of isolating the causes of the improvements. Studies using game engines such as Unity often don't isolate the impact of data layout, instead combining it with other systems or techniques such as Entity component systems and multi-threading. There is also a limited number of studies that compare Object-Oriented programming, Array of Structures, and Structure of Arrays implementations of equivalent rendering and physics workloads within the same C++ game engine, using the same algorithms.

### **2.6 Research Gap**

The existing literature demonstrates that data organisation can influence execution time, cache behaviour and opportunities for SIMD. However, much of the available literature is based on controlled microbenchmarks or specialised numerical workloads. Studies such as Hirzel [11], Kallman [13], and Homann and Laenen [17] examine alternative data layouts under comparatively restrictive conditions. These studies are valuable because they isolate parts of data organisation that influence different aspects such as performance. However, they don't represent the workload of interactions between multiple systems comparable to a game engine.

Research conducted with commercial game engines is more applicable since the workload shows the interaction of different systems within a game engine. However, the studies introduce other architectural implementations that don't isolate the effect of data layout on performance. The entity component system [14] and Unity DOTS [15] both use entity-component systems and multithreading. They do show improvements over the Object-Oriented implementations; however, the additional architectural elements fail to isolate the impact of data layouts on implementation performance, making it unclear how layouts alone affect performance.

The studies are also commonly limited to a single processor or hardware platform [15, 17]. This limits the generalisability of the results to other processors, since cache organisation, memory behaviour, SIMD support and compiler optimisation can differ between architectures. This means there is value in

benchmarking results across different processors and hardware platforms, as the results can be more generalisable. There is also a limited amount of literature that directly compares Object-Oriented Programming, AoS and SoA implementations in a game engine. The existing literature generally focuses on either isolating data-processing kernels or complete data-oriented tech stacks that combine other architectural implementations. Also, within the literature reviewed, limited work directly compares how OOP, AoS and SoA affect distinct systems such as rendering and physics simulation under controlled conditions.

This dissertation will address these gaps by implementing Data-Oriented AoS and SoA into an Object-Oriented game engine framework in C++. The dissertation will keep the algorithms across these implementations equivalent to ensure a fair comparison. These implementations will be evaluated using a consistent workload across processors and varying object counts. This approach makes it possible to examine whether the relative performance of each organisation depends on the engine system, workload size, and the processor on which it is executed, rather than assuming that a single programming or memory-layout approach is universally superior.

### 3 Design and Methodology

#### 3.1 Overview

The engine's baseline was developed using an Object-Oriented Programming paradigm because it was from a previous course (CSC8503). As a result, the AoS and SoA implementations build on the groundwork laid by the Object-Oriented implementation. The implementation contains equivalent algorithms to maintain fairness during testing; the main differences are in how data is laid out in memory, how it is accessed, and who owns the data. However, some changes were made to the Object-Oriented implementation in accordance with the AoS and SoA algorithm changes, such as how data is grouped in the renderer and some changes to the quadtree.

#### 3.2 AoS Implementation

The AoS implementation is based on the engine's initial Object-Oriented implementation. The main design change involves changing how ownership and physical organisation of data are handled. In the initial Object-Oriented implementation, entities (game objects) are represented by separately referenced entities; the AoS implementation stores the complete entity records for each game object in an array. Entities contain properties such as transform, physics, rendering, collision information, and different flags.

```
struct GameObjectDOD {
    PhysicsObjectComp physics;
    TransformsComp transform;
    RenderObjectComp render;
    Vector3 broadphaseAABB;
    AABBComp collision;
    int worldID;
    int collisionLayer;
    bool isActive;
    bool isCollided;
    GameObjectType objectType;
};
```

Figure 3.1: Data stored in GameObjectDOD

This means that entities are stored as consecutive records in memory. Each entity record contains smaller structures such as TransformComp, PhysicsObjectComp, and RenderObjectComp. This allows for a convenient one-record-per-entity structure while reducing the number of separately allocated component objects. This also allows the lifetime of an entity to be controlled through a single storage container, meaning that object creation and removal are centralised GameObjectStorage rather than distributed across individual engine systems. The drawback of this design is that systems must traverse the entire entity to retrieve the desired record. For example, the physics system primarily uses transforms and physics data, whereas the renderer primarily uses transforms and rendering data. Although grouping data into an entity simplifies access, it also means that a small subset required by a specific system must move from the GameObjectDOD to that system. This can lead to degraded performance as structures get larger.

There's a clear separation between data and behaviour, as seen in aspects such as the AoS implementation of transforms. TransformComp stores the data for the transforms, such as the transform matrix, orientation, position, and scale.

```
struct TransformsComp {
    Matrix4 matrix;
    Quaternion orientation;
    Vector3 position;
    Vector3 scale;

    TransformsComp()
        : matrix(Matrix4()), orientation(Quaternion()),
          position(Vector3()), scale(Vector3(1, 1, 1)) {}
};
```

Figure 3.2: Demonstration of TransformsComp, separating data

In contrast, functions are stored in TransformOps, which perform operations on the data, such as performing matrix updates and modifying other transforms, such as position.

## Object-Oriented Programming and Data-Oriented Design: A Performance Comparison

```
namespace TransformOps {  
  
    inline void UpdateMatrix(TransformComp& transform) {  
        transform.matrix = Matrix::Translation(transform.position) *  
            Quaternion::RotationMatrix<Matrix4>(transform.orientation) *  
            Matrix::Scale(transform.scale);  
    }  
  
    inline void SetPosition(TransformComp& transform, const Vector3& worldPos) {  
        transform.position = worldPos;  
        UpdateMatrix(transform);  
    }  
  
    inline void SetScale(TransformComp& transform, const Vector3& worldScale) {  
        transform.scale = worldScale;  
        UpdateMatrix(transform);  
    }  
  
    inline void SetOrientation(TransformComp& transform, const Quaternion& worldOrientation) {  
        transform.orientation = worldOrientation;  
        UpdateMatrix(transform);  
    }  
}
```

**Figure 3.3: Demonstration of TransformOps, separating behaviour**

This is also seen in the implementation of physicsObjectDOD and renderObjectDOD, where the data is stored in its own struct, and the functions are accessed via a namespace that operates on that data. This means that data required by an operation is explicit without requiring each component to provide a large Object-Oriented interface. Although this is a Data-Oriented conversion, that does not necessarily mean that all features of Object-Orientation within C++ have been removed. The use of classes has been retained without inheritance or polymorphism. Classes were used where they represented long-lived system ownership and coordination rather than individual data records. For example, GameWorldDOD owns the central entity storage, camera, and spatial partitioning structure, while PhysicsSystemDOD coordinates the physics update stages and retains a reference to the world.

```
class GameWorldDOD {  
public:  
    GameWorldDOD();  
    ~GameWorldDOD() = default;  
  
    GameWorldData data;  
    GameObjectStorage gameObjects;  
    PerspectiveCamera mainCamera;  
    QuadTreeDOD<size_t> quadTree;  
  
    void AddGameObject(GameObjectDOD& obj);  
    void RemoveGameObject(size_t index);  
    void Clear();  
};
```

**Figure 3.4: Demonstrates the retaining of classes**

This means that the design applies Data-Oriented principles primarily to the representation and processing of frequently accessed data.

Data applied to an entire system is not stored in GameObjectDOD; this avoids repeating global or resource-level values across every entity and groups data by ownership and lifetime. This can be seen in different systems, such as PhysicsSystemDOD; PhysicsSystemData contains global settings for physics, such as gravity, damping, fixed-step accumulation, and collision-persistence configuration.

```
struct PhysicsSystemData {  
    Vector3 gravity;  
    float globalDamping;  
    float dTOffset;  
    bool applyGravity;  
    bool useBroadPhase;  
    int numCollisionFrames;  
  
    PhysicsSystemData()  
        : gravity(Vector3(0.0f, -9.8f, 0.0f)), globalDamping(0.995f),  
          dTOffset(0.0f), applyGravity(false), useBroadPhase(false), numCollisionFrames(5) {}  
};
```

**Figure 3.5: PhysicsSystemData contains data applied to the entire physics system**

Another example of this can be seen in GameTechRendererDOD, where data is separated by use: GameTechRendererResources contains persistent graphics resources such as shaders and buffers. In contrast, GameTechRendererData stores per-frame information such as camera matrices and visible-object lists. Array indices are used to represent relationships between entities. This can be seen in the CollisionDetectionDOD section, where collision contacts store the indices of the two colliding entities; another example is the broadphase implementation in PhysicsSystemDOD, where broadphase-pairs contain the indices of the two colliding entities.

The GameTechRendererDOD also constructs opaque and transparent lists containing indices into the central object array. This means that systems can refer to different entities without copying the complete record into a temporary collection. It also allows a later processing stage to use an index to retrieve the required entity data from GameObjectStorage.

Collision Detection follows the same general separation of data and operations but uses progressively narrower interfaces. ObjectIntersection accepts complete records where high-level volume dispatch is required. AABBIntersection only operates on transform and collision volume components, while AABBTest operate directly on positions and half dimensions. This design prevents lower-level geometric calculations from depending on rendering, gameplay or unrelated physics data. Collision-volume representation was another system-specific design choice. Instead of relying on polymorphism, CollisionVolumeSys uses a type field together with directly stored AABB records. This removes the requirement of independent allocation and allows the volume representation to be embedded directly into the entity record.

The renderer also follows this general design pattern, storing specific data separately based on how it is used. This is because different pieces of data have different lifetimes depending on the context they're used in. Data such as Meshes, shaders, and OpenGL buffers persist across multiple frames, whereas data such as visible-object lists and camera matrices are rebuilt every frame. For this reason, the renderer separates and stores persistent data separately and creates lists containing indices of opaque and transparent entities every frame. The complete entity is still stored in `GameObjectStorage` and is retrieved only when a rendering pass processes the corresponding index.

Overall, the AoS implementation centralises entities as contiguous, complete records while separating system-wide configuration, persistent resources, and temporary relationships by ownership and lifetime. The design reduces reliance on separately allocated entities by storing the complete record of an entity in an array of game objects. This retains simple data access while also retaining a contiguous data layout in memory. However, the main drawback of this approach is that system-specific loops must traverse records that may not be required for the computation in question. This main drawback forms one of the main principles of the SoA approach and implementation.

### 3.3 SoA Implementation

The SoA implementation was based on the engine's initial Object-Oriented implementation. The main design change involves how data is stored when compared to AoS and OOP. AoS stores an entity's data in a complete record, whereas SoA stores each piece of data in its own separate, contiguous array. The conceptual idea of an entity is essentially gone; in traditional thinking, 'entities' are instead represented by a shared index, where element *i* in each associated array refers to the same conceptual entity.

The central representation for this implementation is `GameObjectCompSoA`. Instead of containing an array of complete game object records, this structure contains separate arrays for IDs, object types, flags, collision state, collision layers, and broadphase dimensions.

```
struct GameObjectCompSOA {
    std::vector<int> worldIDs;
    std::vector<GameObjectType> objectTypes;
    std::vector<char> isActive;
    std::vector<char> isCollided;
    std::vector<int> collisionLayers;
    std::vector<Vector3> broadphaseAABBs;

    TransformCompSOA transforms;
    PhysicsObjectCompSOA physics;
    RenderObjectCompSOA render;
    CollisionVolumeSysSOA collision;
};
```

Figure 3.6: Data stored in arrays in `GameObjectCompSOA`

Other data, such as data associated with transforms, physics, rendering, and collision volumes, is stored in separate SoA component containers; for example, `TransformComp` contains transformation matrices, positions, orientations, and scales in separate arrays.

```
struct TransformCompSOA {
    std::vector<Matrix4> matrices;
    std::vector<Quaternion> orientations;
    std::vector<Vector3> positions;
    std::vector<Vector3> scales;
};
```

Figure 3.7: Data stored in arrays in `TransformCompSOA`

This means that high-level object properties are stored in parallel arrays, while each major component contains additional arrays for individual fields.

This design does not form a complete object record that's passed between systems; instead, systems can access arrays containing the properties required by their current operation. All component arrays follow a shared-index invariant. If entity *i* has a position at transforms, `position[i]`, its other properties will be in their own arrays at the same index. Since no object structure binds these properties together, this relationship must be maintained throughout the engine.

The creation and deletion of entities are centralised through `GameObjectOpsSOA`, so that every associated array is modified in a single operation. When an entity is created, the implementation appends the high-level Object arrays and invokes the separate SoA component container functions, such as `TransformCompSOA`. Each component function appends one element to all the parallel arrays; this means that the returned index identifies the newly appended element across the complete storage. This centralised creation process means there is a lower likelihood that a property array is updated without the others being updated.

## Object-Oriented Programming and Data-Oriented Design: A Performance Comparison

This data layout means that individual update stages and systems can access the data specifically needed by that stage or update. For example, the physics system will mainly require physics data, and the acceleration-integration stage will mainly require data such as linear velocity, torque, inverse mass, elasticity, etc. This contrasts with the AoS implementation, since a system must traverse the entire record to access the required data.

Although this allows systems and update stages to access the data they require without traversing unnecessary data, it doesn't necessarily mean performance will be significantly better. This is because calculations that require most of the state to belong to a single entity must access multiple arrays using the same index, increasing the number of distinct memory locations and making entity lifecycle management more complex.

Transform data follows the same design; positions, orientations, scales, and matrices are stored in separate arrays within `TransformCompSoA`, while functions that operate on this data are in the namespace `TransformOpsSoA`. These functions receive the transform storage together with an entity index and explicitly access the required properties. The transformation matrix remains stored as a derived value rather than being recalculated every time it's consumed. Setters update the relevant property and then rebuild the corresponding matrix.

Caching the matrix avoids constantly reconstructing it during rendering; however, this introduces a consistency problem. Positions, Orientations, and scales should be modified using the functions to update the cached matrix accordingly. Modifying an individual array directly without rebuilding the matrix could leave the stored transform state inconsistent. As in the AoS implementation, data and behaviour were separate for the same reasons; this is also true for the use of classes.

Not all data are stored in arrays; some are global because their values are identical across all array indices, making an array unnecessary. Instead of storing properties associated with Boolean operations in arrays of Booleans, these arrays contain chars. This provides ordinary byte-addressable elements and avoids the specialised bit-packed proxy behaviour of a Boolean array. Although this uses more space than individual bits, it simplifies indexed reads and writes and provides storage behaviour consistent with the other property arrays.

The Collision Volume implementation differs slightly from the implementation of the other component stores. Common volume information, such as type or collision layer, is stored in parallel arrays. Shape-specific information is stored separately, with half sizes being stored in their own array. A

`typeDataIndex` array maps a logical collision-volume entry to its corresponding shape-specific data.

This approach avoids reserving storage for every possible collision shape within every entity. This also means that each shape category maintains its own property arrays. The drawback is the added index complexity, as a volume may require both its general entity index and a shape-specific data index. This implementation only supports AABB; however, the separation provides extensibility.

Collision Detection operates directly on the central SoA storage and entity indices. `ObjectIntersection` receives the component storage along with two indices, rather than two complete object records. `AABBIntersection` works similarly by using the indices to retrieve the requested transforms and half-sizes, but the test is `AABBTest`, which receives only primitive positions and dimensions.

This means that higher-level functions coordinate entity lookup and collision-volume dispatch, whereas lower-level geometric routines remain independent of the wider storage representation. Temporary relationship structs such as `CollisionInfoSOA` were not converted to SoA.

```
struct CollisionInfoSOA {
    int entityA;
    int entityB;
    int framesLeft;

    Vector3 localA;
    Vector3 localB;
    Vector3 normal;
    float penetration;
    bool isActive;

    CollisionInfoSOA();
    CollisionInfoSOA(int a, int b);

    void AddContactPoint(const Vector3& lA, const Vector3& lB, const Vector3& norm, float pene);
};
```

**Figure 3.8: CollisionInfoSOA showing the data is not stored in separate arrays**

This is because each record groups the two participating entity indices with the data required by the next process. Collision resolution commonly requires the contact normal, penetration depth, local contact points, and both entity identifiers. This means the design follows the access patterns of each data set rather than forcing all structures to be SoA when unnecessary.

The renderer also separates per-entity rendering properties into parallel arrays. Data such as Mesh pointers are stored in `RenderObjectCompSOA`. These arrays contain pointers to rendering resources rather than the resources themselves. This means that these rendering resources are not stored contiguously. The Renderer is similar to the AoS implementation in that it separates data depending on the lifetime context. The renderer stores indices of opaque and transparent objects rather than copying all associated transform and material data into the lists. Rendering passes use these indices to retrieve the required fields from the central arrays.

Overall, the SoA implementation distributes the properties of each logical entity into separate contiguous arrays, each indexed by a shared index. This means that systems and update stages can access only the required data, rather than potentially traversing an entire entity record. System-wide configuration, persistent graphics resources and temporary collision relationships remain outside the per-entity SoA arrays because their ownership, lifetime or access patterns differ. The main drawback of SoA is the increased complexity of maintaining a consistent index across all arrays, which becomes more challenging when adding or removing entities.

### **3.4 Shared Implementation Changes**

Several changes were made to each implementation to optimise algorithms, show the impact of optimisation on each implementation, and assess how scalable those optimisations will be. These changes were applied to all three implementations to ensure a fair comparison; the algorithms also logically maintain equivalence with one another. Each implementation will organise data differently, reflecting the differences of the implementations.

The renderer used equivalent object classification and render-state reduction techniques across implementations. Opaque objects were ordered according to their texture and mesh resources, allowing objects that used the same resources to be processed consecutively. Rendering passes also retained the previously bound texture and mesh, thereby avoiding redundant OpenGL state changes. The same rendering behaviour and resource-ordering objectives were retained across all implementations. The physics system maintains a collection of dynamic-object indices to avoid processing entities that could not move.

This change was made across all implementations. The broadphase quadtree was also revised across the implementations. The original tree recursively inserted an object into every child node whose region intersected its bounding box. This could place one object into multiple nodes, increasing the number of stored entities and potentially generating duplicate candidate pairs. The revised design places an object into a child only when its complete axis-aligned bounding box is contained within that child. Objects spanning more than one child remained at the current node.

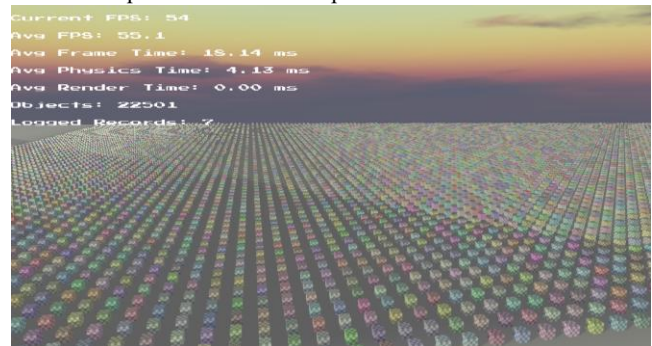
This retained a single spatial entity per object, reducing duplication during subdivision. The tree is rebuilt only when object movement invalidates the existing spatial partition. A dirty flag is set after position integration, and the tree is cleared and repopulated before the next broadphase evaluation. Dynamic objects are inserted into the tree, while static-dynamic candidate pairs are evaluated separately using the same proximity condition in each implementation.

These were shared changes made across the engine; they are logically the same algorithms, and the differences between them reflect how each layout and accesses data. The OOP implementation follows object and component references; the AoS implementation traverses complete entity records; and the SoA implementation accesses parallel property arrays via shared indices.

### **3.5 Benchmark Setup**

This benchmark was designed to evaluate how data layout and data access patterns affect the performance of the OOP, AoS, and SoA game-engine implementations. Each implementation uses equivalent algorithms for physics and rendering, with the main difference being the ownership, layout, and method of accessing per-entity data.

The benchmark scene consists of a static plane and a grid of identically sized cubes. Each cube has the same physics properties; the only difference in the rendering properties is the colour of each cube; every other property is the same. At the beginning of the test, the cubes are positioned above the plane and fall under the influence of gravity. The cubes then collide with the plane and rest on the plane with a restitution of 0.66.



**Figure 3.9: Benchmark scene**

Four workloads were selected for the comparison. These consist of object counts: 101, 2501, 10001, 40001. The extra one includes the static floor; the remaining objects are the dynamic cubes. The same object positions, dimensions, physics properties, rendering resources, and camera configuration were used across all three implementations. This was done to ensure that different scene configurations did not cause performance issues. The physics configuration uses a fixed time step of 1/60 seconds, and the broadphase quadtree has a world size of 1024 x 1024, a maximum depth of seven, and a node capacity of ten. These parameters were kept consistent across the implementations. Each benchmark was executed for 30 seconds to capture the changing workload as the cubes fall to the ground and bounce.

Performance measurements were recorded in one-second time intervals throughout the execution of the implementation. The resulting time-series graphs show how performance changes

over the execution time. The benchmark was conducted on two separate PCs that contain different processors, GPUs, and memory configurations. The PCs execute the implementations on the same Windows 11 operating system and use the same MSVC compiler featured in Visual Studio 2022. The same executable, configuration, and benchmark procedure were used across all implementations on each PC.

Four Performance metrics were recorded during the benchmark: Frame time, FPS, Physics Timing, and Rendering Timing. The FPS represents the number of frames captured in a single one-second time interval; the frame time represents the mean duration of a completed frame. Physics timing represents the mean CPU time spent executing the physics system update; rendering timing represents the same concept for the rendering system update. Frame time is treated as the primary measurement because it represents overall performance by directly reflecting the time required to execute a frame. FPS was also recorded as it is commonly used to determine the performance of real-time applications. These were treated as two independent measurements because they represent the same performance relationship. Physics timing was measured immediately before and after the physics-system update; the same was true for render timing. These measurements were written to a CSV file that puts the values in a table.

The procedure for this benchmark is as follows: MSVC compiled implementations in release mode. The benchmark scenario was launched with no loading required, with the desired implementation being tested and the desired workload.

The implementation was run for 30 seconds, during which the measurements were written to a CSV file every second. After the 30-second interval, the implementation and software were closed.

The CSV files were processed using Python and matplotlib to record the results onto graphs. Results are graphed against the elapsed time to show the change throughout the implementation. Only one run was executed for each implementation and workload; this means that the results are interpreted descriptively. They show the performance observed under the selected test conditions but do not measure between-run variability or establish statistical significance. This is a limitation as performance measurements can vary between executions. Hocko and Kalibera demonstrate that page allocation behaviour can contribute to performance non-determinism by altering cache interactions [20].

Another experiment was also conducted to illustrate the advantages of SoA artificially. Both the SoA and AoS implementations are given unused values; in AoS, the unused

values increase the size and stride of each entity record. In SoA, the equivalent values are stored in separate arrays. The Benchmark procedure and recorded metrics are the same as in the initial experiment.

## 4 Results and Evaluation

### 4.1 AMD Ryzen 9 7900X 12-core results

The results presented were captured on an AMD Ryzen 9 7900X 12-core processor; other processors will also be evaluated and compared to conclude the layout's relationship to the hardware.

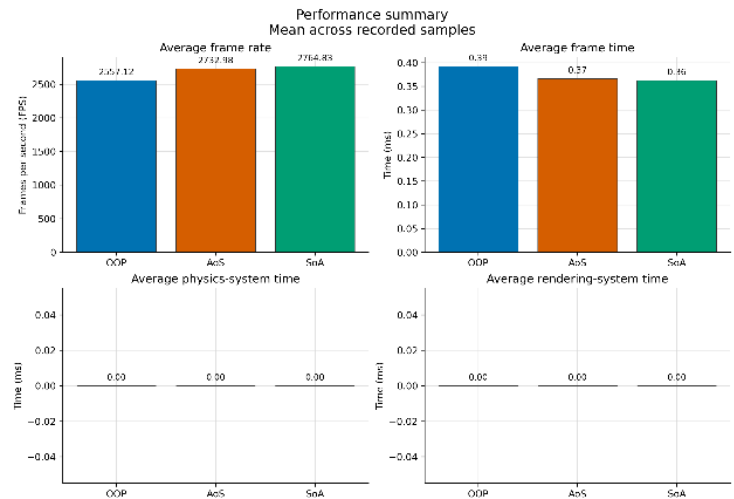
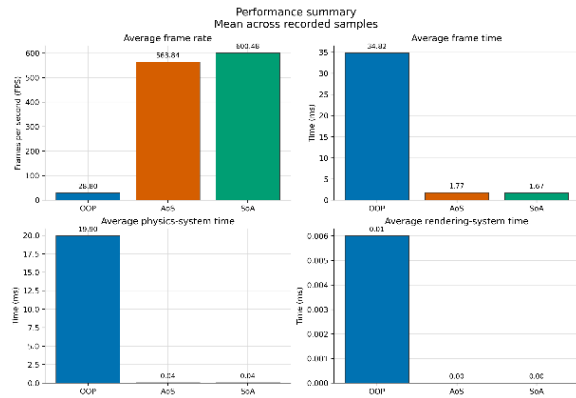


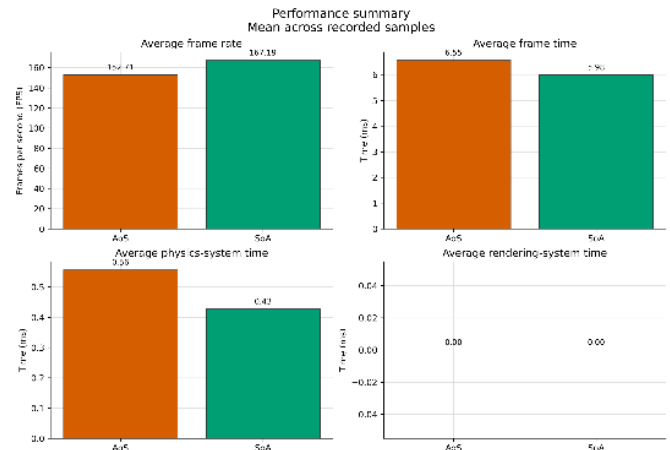
Figure 4.1: Charts showing the averages of the measurements with the object count of 101

The results of the first workload show similar performance across the three implementations. Each implementation shows an average frame time below 0.4ms. While SoA performed better than AoS, the improvement was only marginal, with an approximate difference of 0.01ms, corresponding to an overhead reduction of approximately 2.7% in frame time. The physics time and rendering time recorded were 0, revealing that these systems' overhead doesn't influence performance at lighter workloads. At this workload, the benefits of data orientation don't necessarily reveal themselves when compared to object orientation; this means that when performing operations on lighter workloads, data layout and access may not be deciding factors in influencing the performance of game-engine systems.



**Figure 4.2:** Charts showing the average measurements of the implementations with an object count of 2501

Figure 4.4.2 stands in contrast to Figure 4.4.1; the larger workload now shows clear distinctions and benefits in how data is laid out and accessed in memory. This can be seen in the average frame time of each implementation; OOP achieves an average frame time of 34.82 ms, whereas the frame times of AoS and SoA are significantly lower, at 1.77 ms and 1.67 ms, respectively. Physics timing is also a key measurement for comparing how much more performant AoS and SoA are compared to OOP: OOP achieves a physics time of 19.90ms, compared to 0.4ms for both AoS and SoA. This is because of how data is laid out in and accessed in memory. The OOP implementation constantly employs pointer indirection and polymorphism throughout the implementation, leading to a scattered memory layout which is further worsened by inheritance, which adds to values that aren't required by systems being put onto cache lines. This stands in contrast to AoS and SoA, which decouple data from class encapsulation, meaning that systems get the data required for their computation. This reduces the number of separately allocated per-object components and centralises entity data within contiguous vector storage. Similarly to Figure 4.4.1, SoA only shows a marginal improvement over AoS with a frame time that is approximately 5.6% lower than AoS. This does show, however, that as workloads increase, SoA performs increasingly better than AoS.



**Figure 4.3:** Charts showing the average measurements of the implementations with an object count of 10001

At this workload, the baseline OOP implementation cannot record results to compare against AoS and SoA because it constantly stalls and has severe frame delays, causing inaccurate frame time and an inability to log performance metrics to the CSV file. As a result, the OOP results for this workload and the last workload are N/A. The trend of SoA only performing marginally better than AoS from Figure 4.4.1 and Figure 4.4.2 remains true with this workload as well; the results have an average frame time of 6.55ms for AoS and an average frame time of 5.98 for SoA, which is approximately 8.7%. However, the physics timing difference between the two implementations shows that SoA has better physics system CPU performance. The physics time for AoS is 0.56 ms, whereas the result for SoA is 0.43 ms; this is approximately a 25.9% reduction, which is a significant difference in performance. This suggests that the memory access pattern present in SoA does have a significant influence on system performance; this could be because the physics system is only accessing data that is required for physics computation. In contrast, AoS requires the physics system to traverse complete records, which include data that is not necessarily required for that specific computation.

## Object-Oriented Programming and Data-Oriented Design: A Performance Comparison

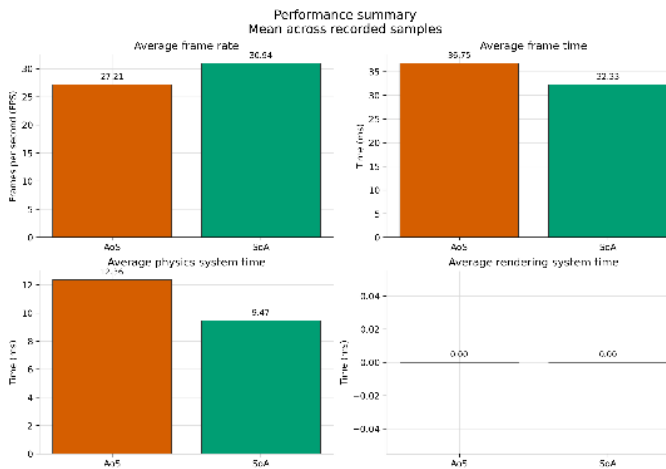


Figure 4.4: Charts showing the average measurements of the implementations with an object count of 40001

At this workload, AoS and SoA are unable to maintain a 16.67 ms or below frame time; instead, AoS has an average frame time of 36.75 ms, and SoA has an average frame time of 32.33 ms. This means that SoA has a reduction of approximately 12.00% relative to AoS; this suggests that SoA has an advantage over AoS in traversing data, which becomes more beneficial as the workload increases. This is also evident in the physics time, as it continues to show that physics times are consistently less than AoS, suggesting that the physics system is taking advantage of the access pattern inherent to SoA compared to AoS.

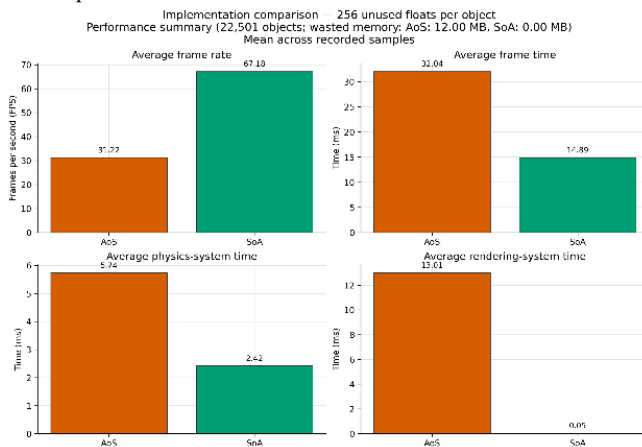


Figure 4.5: Charts that show the averages of the implementations when memory is being wasted

Figure 4.4.5 shows the result of the unused data experiment. The results show the benefits of SoA when the size of entities increases. The average frame time of AoS is 32.04 ms, whereas the average frame time of SoA is 14.89 ms; that's a reduction of approximately 53.5% relative to AoS. This can

also be seen in both the average physics time and average rendering time, where AoS has an average time of 5.74 ms and the average physics time for SoA is 2.42 ms. The average rendering time for AoS is 13.01 ms, and the average rendering time for SoA is 0.5 ms. This suggests that as entities contain more properties, systems spend more time traversing records to retrieve data required for their specific computation. Whereas SoA allows systems to retrieve only required data for computation, even with unused data present in the SoA implementation, the systems didn't access it since those unused values are present in their own separate array that doesn't get accessed if not required, instead of being stored in a complete record of an entity.

### 4.2 13<sup>th</sup> Gen intel(R) Core™ i7-13650HX – 14 Core Processor

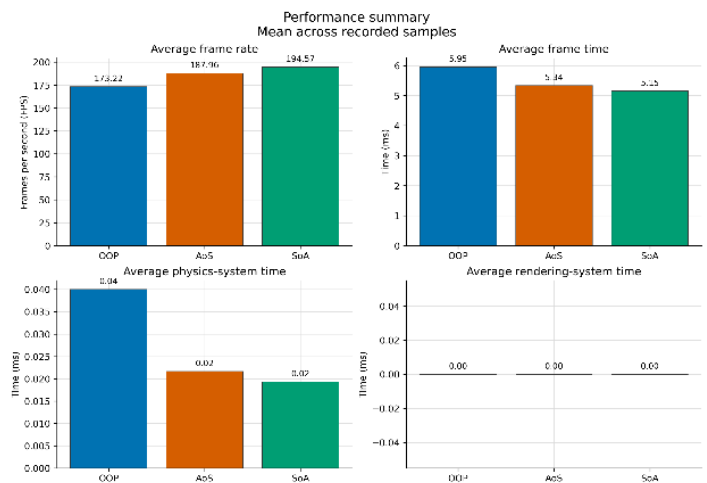
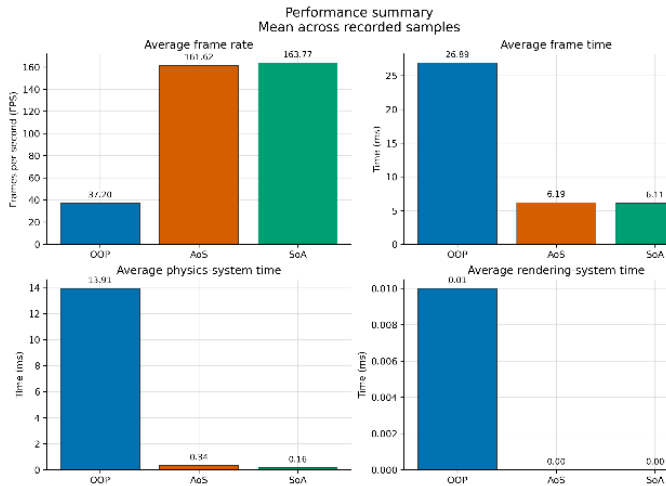


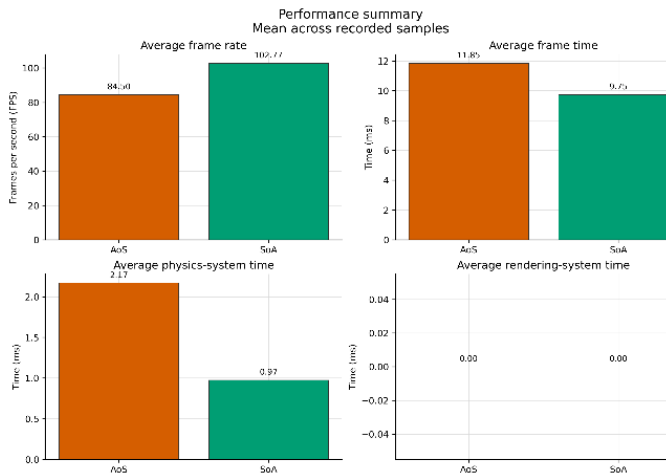
Figure 4.6: Chart that shows the averages of the implementations (101 Objects, Laptop)

Figure 4.6 shows the light workload executing on Intel i7-13650HX. The results are similar to the Ryzen PC; however, the main difference here is that physics timing is being recorded, which was not true on the Ryzen PC. The subsystem timings differed between the two hardware platforms. However, the benchmark did not isolate processor frequency, cache organisation, memory behaviour, graphics hardware or system-level effects, so the cause of this difference cannot be determined from the recorded measurements alone. Also, SoA (5.15 ms) saw a 3.5% decrease when compared to AoS (5.34 ms). This contrasts with Figure 4.4.1, which shows a decrease of 2.7%; however, this isn't a big enough difference to make any suggestions.



**Figure 4.7:** Charts showing the average measurements of the implementations with an object count of 2501 (Laptop)

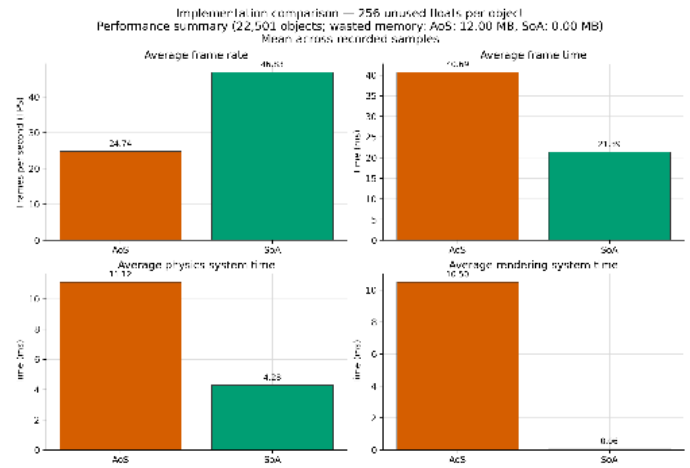
The main difference in this workload is the smaller frame time of OOP in comparison to Figure 4.4.2. Figure 4.7 shows that OOP recorded an average frame time of 26.89 ms, whereas Figure 4.4.2 shows OOP recorded an average frame time of 34.82; that's a 22.77% decrease in frame time. Additionally, the physics time in Figure 4.7 is lower than in Figure 4.4.2; this could suggest that operations primarily associated with OOP, such as pointer-based object access, may vary depending on CPU architecture.



**Figure 4.8:** Charts showing the average measurements of the implementations with an object count of 10001 (Laptop)

Figure 4.8 presents a greater difference between SoA and AoS when compared to Figure 4.4.3. Figure 4.8 records that SoA had a frame time of 9.75 ms and AoS had a frame time of 11.85 ms. This may be explained by the greater reduction in physics time when compared to Figure 4.4.3. Figure 4.8 shows a reduction of 55.30% in physics time, compared to Figure 4.4.3, which shows a reduction of 23.21% in physics time.

This could suggest that physics operations are more dependent on CPU architecture than data layout and access alone. This platform couldn't run the heavy workload scenario, so they can't be compared.



**Figure 4.9:** Charts that show the averages of the implementations when memory is being wasted (Laptop)

There isn't much of a change in performance when compared to Figure 4.5; the decrease in frame time and physics time is close between the two platforms, relative to the hardware present on them. This could suggest that data layout and hardware equally influence the performance of game systems since there isn't much of a change in the decrease relative to the hardware platform.

### 4.3 Evaluation

The results suggest that not only does Data-Oriented Design have many advantages in terms of performance over Object-Oriented Design, but they also suggest that data layout and access patterns are very important, and negligence of this area can lead to poor performance of game systems, such as CPU-oriented systems like physics.

Evaluating the results of these experiments against the research questions asked, the results suggest that Data-Oriented Design differs from Object-Oriented Programming by focusing on how data is shaped, transformed, and accessed. In contrast, Object-Oriented Programming focuses on world modelling and modular systems through polymorphism and inheritance, at the cost of data layout that promotes efficient use of the cache hierarchy and cache lines through spatial locality. This can be seen in the results; for example, AoS and SoA consistently outperform OOP in all metrics and go beyond the capabilities of the baseline OOP implementation, even with optimisations being applied to all implementations. AoS and SoA scaled better than OOP, allowing more objects to be loaded with different physics and rendering properties, and consistently

producing low frame times, physics times, and rendering (CPU side) times.

The results can also be used to evaluate how AoS and SoA affect game-engine performance. The results show that SoA consistently outperformed AoS (marginally) and showed the advantages at larger object counts. Initially, the benefits of SoA were very small improvements over AoS. Still, when the workload reached 40001, the frame time decrease from AoS to SoA increased to 12%, showing that SoA has performance benefits with larger workloads compared to AoS. This is because of the access patterns and data layout inherent to SoA, allowing systems to access required data for the system-specific computation directly. This was also shown in the unused data experiment, which shows how AoS struggles when entity records become bigger; this is because systems have to traverse entity records that may not always contain specific data for specific computation. SoA doesn't require systems to traverse entity records and instead provides direct access to data through contiguous data layouts in the form of arrays. Both SoA and AoS have advantages that can affect game-engine performance in many different ways.

The results can also be used to evaluate what game engine systems benefit from adopting DataOriented Design. The results show that the physics system saw the most impact when compared to the renderer. This can be seen with higher workloads, where the physics system has much more use than the renderer. However, the unused data experiment does show that, if required properties are accessed, the renderer does benefit from the data access that is inherent to SoA's contiguous data layout. The reason why the physics system benefits the most from DOD when compared to the renderer is that physics are entirely done on the CPU, whereas the renderer does CPU-bound calculations that are then sent to the GPU. If the renderer were a software renderer, there would be more CPU usage for the renderer. This also suggests that other, traditionally CPU-bound systems such as Artificial Intelligence could see similar performance benefits from Data-Oriented Design that the physics system does.

Overall, the results show that Data-Oriented Design does have performance benefits when compared to Object-Oriented Programming because of the focus on data transformations, data layout, data access, and efficient use of cache lines and cache hierarchy. This contrasts with Object-Oriented Programming, which focuses on modularity and abstraction through encapsulation, inheritance, and polymorphism at the cost of data layout and data access. The results also show that neither AoS nor SoA are universally superior under every condition; for example, the benefits of SoA were limited with smaller workloads but became more apparent with larger workloads or when accessing selected properties only from large collections, as was the case with the unused data

experiment. However, these conclusions are limited because of the lack of repeated tests with the different implementations and workloads and therefore should be treated as observed benchmarks.

## **5 Conclusion**

In conclusion, this research into Data-Oriented Design, memory layout and access, and computer architecture has demonstrated the benefits of DOD by comparing an Object-Oriented implementation and the performance metric produced from each implementation. This research has also shown the benefits of and impact of data layouts and access through the implementation of AoS and SoA structures. AoS stores the properties of entities in complete records and allows systems to access the data required for system-specific computation, at the cost of systems having to traverse records containing data that may not be required. SoA stores properties of entities in separate contiguous arrays, allowing systems to access data without having to traverse entity records that may contain data that is not required. AoS retains a simple structure while remaining performant, whereas SoA sacrifices some simplicity to achieve contiguous memory layouts and direct memory access without wasting cache line space. These design choices inherent to DOD and the memory layout of AoS and SoA make it easier to allocate memory on the stack rather than heap-allocating objects because of pointer chasing and pointer indirection that is prevalent in OOP implementations, including the baseline engine implementation.

### **5.1 Future Work**

Although the research has achieved the different goals and answered the different questions posed at the beginning of the research, there are still different areas for improvement in the implementation and the testing methodology. A primary feature that could be added in future work is a fully multi-threaded implementation applied across all implementations. While this would mean that data layout and data access aren't isolated as primary influences over performance, it does mean that the behaviour of data layouts can be better leveraged, such as SoA. Since systems only access required data, different systems could access different data and perform their specific computation at the same time across multiple threads; this would only enhance the SoA implementation. This would also be beneficial for the other implementations but to different degrees, since OOP still has scattered data layouts and data access. In contrast, AoS has contiguous storage of entity records, but systems are still required to traverse records of data that is potentially not required. This also shows how extendable DOD is compared to OOP, since multithreaded conversations are usually simpler in DOD. Studies cited have also shown performance improvements with DOD implementations; for Unity [15] and SoA [14], this could make the research even more conclusive.

Another aspect that could make the research more conclusive in the context of game development would be to use the engine to make an actual game. Although the implementation does represent the computation that is apparent in a game engine, it could be argued that it isn't emblematic of the computation that takes place in a game. For example, a game usually also computes systems such as Artificial Intelligence, Audio, Networking, etc. These aspects of a game are absent in the implementation of AoS and SoA and therefore can't necessarily be compared to the computation of a game. This could also be an opportunity to show the benefits of DOD's simplicity when implementing aspects such as networking and AI, where pointer indirection and heap allocation aren't as apparent when compared to many OOP implementations.

Currently, the AoS and SoA renderer uses the OpenGL implementation; however, there is a Vulkan implementation that would be better for AoS and SoA. This is because Vulkan is much lower level than OpenGL, allowing for more control in how data is laid out and accessed in memory. This could lead to much better performance for AoS and especially SoA because of the layout and access of data that's inherent to SoA; the Vulkan renderer would only access rendering data rather than having to traverse any records that don't contain any required data.

Another aspect that could make the benefit of DOD more apparent would be the reduction of booleans in the AoS and SoA implementations. Booleans have a size of 1 byte; however, this can come at the cost of memory alignment. This is because natural alignment has to be taken into consideration. Andrew Kelly talks about this in his presentation, where he shows how Booleans can affect the size of a struct in memory. Kelly shows that a struct containing a u32 and a bool has a natural alignment of 4 and a size of 8 [24]. This is because the u32 fills the first part of the struct with 4 bytes, and then the bool is added and is stored in the second part of the struct; however, that part still needs to be allocated in memory, so there are 3 slots where no data is stored.

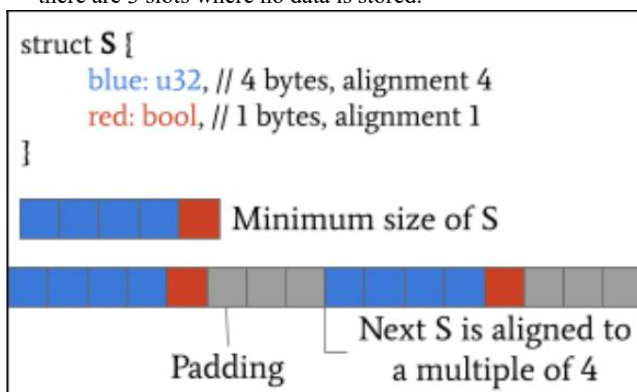


Figure 5.1: Example of memory padding due to boolean [13]

This hinders memory alignment and can affect performance. Kelly proposes that this could be solved by storing an object's invariants in their own arrays [24]. For example, instead of having a boolean that tracks if an object is alive or dead, there could be an array of alive objects and an array of dead objects. This would further show the advantages of DOD design principles.

## References

1. *Design Patterns* - Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). *Design Patterns*. Pearson Education.
2. *The C++ Programming Language* - Bjarne Stroustrup and Addison-Wesley (1942-1999 (2018). *The C++ programming language*. Upper Saddle River: Addison-Wesley, Cop.
3. *Clean Architecture* - Martin, R.C. (2017). *Clean Architecture*. Prentice Hall.
4. Gregory, J. (2019). *Game engine architecture*. Boca Raton, FL: Crc Press, Taylor & Francis Group.
5. Nystrom, R. (2014). *Game programming patterns*. United States: Genever Benning.
6. Stokes, J. (2015). *Inside the machine: An illustrated introduction to microprocessors and computer architecture*. No Starch Press.
7. Fabian, R. (2018). *Data-oriented design: software engineering for limited resources and short schedules*. Place Of Publication Not Identified: Richard Fabian.
8. Hennessy, J.L. and Patterson, D.A. (2019). *Computer Architecture: a Quantitative Approach*. Cambridge, MA: Morgan Kaufmann.
9. *Computer Systems: A Programmer's Perspective* - Bryant, R.E. and O'Hallaron, D.R. (2016). *Computer systems : a programmer's perspective*. Boston: Pearson.
10. Drepper, U. (2007) *What Every Programmer Should Know About Memory*. Red Hat, Inc.
11. Hirzel, M. (2007). Data layouts for object-oriented programs. *ACM SIGMETRICS Performance Evaluation Review*, 35(1), pp.265–276. doi:10.1145/1269899.1254915.
12. Kandemir, M., Ramanujam, J. and Choudhary, A. (1999). Improving cache locality by a combination of loop and data transformations. *IEEE Transactions on Computers*, 48(2), pp.159–167. doi:10.1109/12.752657.
13. Källman, T., 2025. Impact of data-oriented design principles on performance: An experimental study of hardware performance metrics of data-oriented design.
14. Tasnim, A. and Zhao, T. (n.d.). The Essence of Entity Component System. [online] doi:10.1145/3748522.3779910.
15. Martin, B., Visci, J., Visci, M., Thompson, J. and Hodson, D. (2026). Benchmarking Performance of Unity's Data Oriented Technology Stack.

- Communications in Computer and Information Science*, pp.516–526. doi:10.1007/978-3-032-13056-3\_40.
16. Chilimbi, T.M., Hill, M.D. and Larus, J.R. (1999). Cache-conscious structure layout. *ACM SIGPLAN Notices*, 34(5), pp.1–12. doi:10.1145/301631.301633.
  17. Homann, H. and Laenen, F. (2018). SoAx: A generic C++ Structure of Arrays for handling particles in HPC codes. *Computer Physics Communications*, 224, pp.325–332. doi:10.1016/j.cpc.2017.11.015.
  18. Chen, J, Varbanescu, AL & Naumann, A 2025, Optimizing Memory Access Patterns through Automatic Data Layout Transformation (Work in Progress Paper). in *ICPE Companion 2025: Companion of the 16th ACM/SPEC International Conference on Performance Engineering*. ICPE Companion 2025 - Companion of the 16th ACM/SPEC International Conference on Performance Engineering, Association for Computing Machinery, pp. 47-53, 16th ACM/SPEC International Conference on Performance, ICPE 2025, Toronto, Canada, 5/05/25. <https://doi.org/10.1145/3680256.3722203>
  19. Hocko, M. and Kalibera, T. (2010). Reducing performance non-determinism via cache-aware page allocation strategies. *Proceedings of the first joint WOSP/SIPEW international conference on Performance engineering*, pp.223–234. doi:10.1145/1712605.1712640.
  20. Mei, G. and Tian, H., 2014. Performance impact of data layout on the gpu-accelerated idw interpolation. *arXiv preprint arXiv:1402.4986*.
  21. Intel Corporation (2024) *Intel® 64 and IA-32 Architectures Optimization Reference Manual: Volume 1*. Document No. 248966-050US. Santa Clara, CA: Intel Corporation. Available at: [Intel® 64 and IA-32 Architectures Optimization Reference Manual Volume 1](#)
  22. Advanced Micro Devices, Inc. (2024) *Software Optimization Guide for the AMD Zen5 Microarchitecture*. Publication No. 58455, Revision 1.00. Available at: [AMD Documentation Portal](#)
  23. CppCon 2014: Mike Acton “Data-Oriented Design and C++,” (2014). *YouTube*. Available at: <https://www.youtube.com/watch?v=rX0ItVEVjHc>
  24. Handmade Cities (2021). *Andrew Kelley - Practical DOD*. [online] Vimeo. Available at: <https://vimeo.com/649009599>